

# GodZillaKilla — Bug Report and Fixes

NinjaTrader 8 | Strategy silently disabled mid-session | 29 July 2026

|               |                                                                                |
|---------------|--------------------------------------------------------------------------------|
| Component     | bin/Custom/Strategies/Playr101/GodZillaKilla.cs                                |
| Order mode    | ATM Strategy (template MNQLADDER3.PANAZ, 8 contracts, 3 brackets 3/3/2)        |
| Instrument    | MNQ SEP26                                                                      |
| Accounts      | Sim101, Sim102 (NinjaTrader internal simulator)                                |
| Evidence base | NT8 log/ and trace/ files, 24–29 July 2026                                     |
| Defects found | 4 (3 High severity, 1 Low severity)                                            |
| Status        | All 4 fixed, compiled, and verified across a 5-hour live session (130 entries) |

## Executive summary

On two separate occasions on 29 July 2026 the GodZillaKilla strategy was silently disabled by NinjaTrader in the middle of a trading session, with an open position force-closed at market. The same signature appears five times on 24 July and twice on 28 July. No user action, no error and no connectivity event was involved — the strategy shut itself down through a chain of three interacting defects in its own naked-position watchdog.

The root cause is a pair of false positives in `CheckForNakedPositions()`: the watchdog concluded that a live, fully-bracketed position had no protective orders, and reacted by calling `Account.Flatten()`. On NinjaTrader 8, `Account.Flatten()` does not only close the position — it also disables every NinjaScript strategy running on that account. That side effect turned a spurious risk check into a full trading halt that went unnoticed until the user manually re-enabled the strategy hours later.

A fourth, non-destructive defect floods the NT8 log with ATM API errors on every entry, which had been masking the real events and made post-mortem analysis considerably harder.

## Impact observed

- 29 Jul, 06:50:54 — Sim102, instance 383416810 disabled. Open short force-closed.
- 29 Jul, 08:37:44 — Sim101, instance 383416807 disabled 314 ms after an 8-contract entry filled, before its brackets existed. Position closed at a ~\$48 loss with no stop and no target ever placed.
- In both cases trading stopped for the rest of the session (until 13:47 and 13:48 respectively).

## The amplifying factor: `Account.Flatten()` disables the strategy

This is what turns a benign false positive into a session-ending event, and it is worth stating separately because it is easy to miss when reading the code.

Every single time `Account.Flatten()` is called, the NT8 log shows the strategy being disabled in the **same millisecond**. This holds for 8 out of 8 occurrences across the 24–29 July logs, with no exceptions:

```
2026-07-29 08:37:44:850|1|64|Flatten account='Sim101'
2026-07-29 08:37:44:850|1|4 |Disabling NinjaScript strategy 'GodZillaKilla/383416807'
```

In GodZillaKilla, `Account.Flatten()` is reachable from exactly one place:

`FlattenNakedAccountPosition()`. Note that `FlattenEverything()` — used by the daily loss/profit limits, the trading-window close and the CLOSE ALL button — does **not** call it, and therefore never disables the strategy. Only the watchdog path does.

Consequence: any false positive in the watchdog is not a minor annoyance. It is a silent, unrecoverable stop of automated trading for the rest of the session.

## Bug #1 — Transitional order states counted as "no protection"

|            |                                                    |
|------------|----------------------------------------------------|
| Severity   | <b>High</b> — unwanted flatten + strategy shutdown |
| Location   | HasWorkingProtectiveOrders()                       |
| Reproduced | 29 Jul 06:50:54.803, Sim102                        |

The helper decides whether a live position is protected by scanning the account orders. It accepted only three order states:

```
if (o.OrderState != OrderState.Working
    && o.OrderState != OrderState.Accepted
    && o.OrderState != OrderState.Submitted)
    continue; // not counted as protection
```

The problem is that a healthy ATM bracket spends a meaningful amount of time in *other* states. When one leg of an OCO group partially fills, NinjaTrader immediately issues a quantity change on the sibling order, which moves it to ChangePending and then ChangeSubmitted. Meanwhile the filling leg itself sits in PartFilled. During that window — a completely normal part of the bracket lifecycle — the old code saw zero protective orders.

### Evidence

```
06:50:54:797 Target3 New state='Partially filled' Filled=1 Qty=2
06:50:54:798 Stop3 New state='Change submitted' Qty=1 <-- OCO qty adjust
06:50:54:799 Position Sim102 Quantity=1 Short
06:50:54:801 AtmStrategy.CloseStrategy id='383416980'
06:50:54:803 Flatten account='Sim102'
06:50:54:803 Disabling NinjaScript strategy 'GodZillaKilla/383416810'
```

Six milliseconds elapse between the partial fill and the flatten. The remaining 1-lot short position was fully protected the whole time — the stop simply happened to be mid-adjustment.

### Fix applied

Accept the transitional states as valid protection. CancelPending is included because during an OCO transition the replacement order is already in flight; the double-confirmation added in Bug #3 covers the case where the cancel turns out to be final.

```
if (o.OrderState != OrderState.Working
    && o.OrderState != OrderState.Accepted
    && o.OrderState != OrderState.Submitted
    && o.OrderState != OrderState.ChangePending // NEW
    && o.OrderState != OrderState.ChangeSubmitted // NEW
    && o.OrderState != OrderState.CancelPending // NEW
    && o.OrderState != OrderState.PartFilled) // NEW
    continue;
```

## Bug #2 — No grace period between entry fill and bracket submission

|            |                                                    |
|------------|----------------------------------------------------|
| Severity   | <b>High</b> — unwanted flatten + strategy shutdown |
| Location   | CheckForNakedPositions()                           |
| Reproduced | 29 Jul 08:37:44.850, Sim101                        |

In ATM mode the entry order and its brackets are not submitted atomically. NinjaTrader fills the entry first, and only then submits Stop1/Target1/Stop2/... There is therefore a window in which the account genuinely holds a position with no protective orders — not because anything is wrong, but because the brackets have not been sent yet.

The watchdog runs on a 30-second timer driven by market ticks, so it lands at an essentially random moment. On 29 July at 08:37:44 it landed inside that window.

### Evidence

```
08:37:44:269 Entry Market Buy 8 submitted
08:37:44:522 Entry New state='Filled' Filled=8 @ 27811.875
08:37:44:836 AtmStrategy.CloseStrategy id='383416997' <-- watchdog fires (+314 ms)
08:37:44:850 Flatten account='Sim101'
08:37:44:850 Disabling NinjaScript strategy 'GodZillaKilla/383416807'
08:37:45:228 Close New state='Filled' 8 @ 27808.84 <-- ~$48 loss
```

For reference, the immediately preceding cycle shows what the timing normally looks like: entry filled at 08:34:31.005, Stop1 submitted at 08:34:31.356 — a 351 ms gap. The watchdog fired 37 ms before the brackets would have appeared.

### Measured bracket latency

The fill-to-Stop1 delay was measured across **130 entries** over a five-hour session on 29 July (14:35–19:43). The distribution is strongly bimodal, which matters for choosing the guard value:

| Samples | Min  | Median | p90    | p95    | Max    | Mean   |
|---------|------|--------|--------|--------|--------|--------|
| 130     | 0 ms | 4 ms   | 422 ms | 457 ms | 644 ms | 136 ms |

Most entries get their brackets almost instantly (median 4 ms), but roughly one entry in ten takes 400 ms or more. **The median is therefore useless for sizing the guard** — only the tail matters, and the tail is where the two production incidents landed. This also explains why the failure was intermittent rather than systematic: the watchdog had to hit a 30-second-random sample inside a several-hundred-millisecond window that itself only occurs on ~10% of entries.

### Fix applied

Timestamp every fill on the instrument and suppress the naked check for 5 seconds afterwards (7.8× the worst observed bracket latency). Fills are stamped in two places, because ATM fills do not reach `OnExecutionUpdate()` — they arrive through the account-level execution hook:

```
// OnExecutionUpdate(...) and OnMarkerAccountExecutionUpdate(...)
_lastFillActivityUtc = DateTime.UtcNow;

// CheckForNakedPositions(...)
if ((DateTime.UtcNow - _lastFillActivityUtc).TotalSeconds < NakedGraceSeconds)
{
    _nakedSuspectPending = false;
    return;
}
```

## Bug #3 — Irreversible action taken on a single unconfirmed observation

**Severity** High — design weakness that amplifies #1 and #2

**Location** `CheckForNakedPositions()`

The watchdog flattened the account and killed the strategy on the strength of one instantaneous observation, in a code path where the observation is inherently racy (orders in flight, ATM state propagating asynchronously). Both defects above would have been harmless with a second look.

### Fix applied

The first sighting no longer flattens. It arms a suspect flag, pulls the next check forward from 30 s to 5 s, and logs a distinct message. The destructive action happens only if the position is still unprotected on the second consecutive look:

```

if (!_nakedSuspectPending)
{
    _nakedSuspectPending = true;
    lastNakedCheckUtc = DateTime.UtcNow
        .AddSeconds (NakedReCheckSeconds - NakedCheckIntervalSeconds);
    Print ("... NAKED POSITION SUSPECTED ... re-checking in 5s before flattening.");
    return;
}
_nakedSuspectPending = false;
Print ("... NAKED POSITION CONFIRMED ... FLATTENING IMMEDIATELY.");
FlattenNakedAccountPosition (...);

```

Worst-case cost: a genuinely unprotected position stays exposed ~5 s longer than before. Given that the watchdog only polls every 30 s anyway, this is well within the existing detection granularity. The flag is also cleared on the disable/re-enable cycle, since NT8 reuses the same C# instance.

## Bug #4 — ATM position polled before AtmStrategyCreate has registered the ID

|           |                                                                         |
|-----------|-------------------------------------------------------------------------|
| Severity  | <b>Low</b> — no trading impact, but heavy log noise                     |
| Location  | GetAtmStrategyMarketPositionTickCached(), TryGetAtmMarketPositionSafe() |
| Frequency | Every entry — 6 to 12 identical errors per entry                        |

atmStrategyId is generated by GetAtmStrategyUniqueId() and stored *before* the asynchronous AtmStrategyCreate() call registers it inside NinjaTrader. Any position query issued in that window targets an ID that NT8 does not know about yet:

```

08:37:44:268 ERROR: 'GetAtmStrategyMarketPosition' method error:
                ATM strategy ID '03759c44cb824cfa9abd7b8798521592' does not exist
                ... the same line repeated 11 times within 1 ms ...
08:37:44:269 Cbi.Account.CreateOrder: orderId='78236ff2...' name='Entry' quantity=8

```

**The key detail**, and the reason the existing defensive wrappers never caught this: NinjaTrader does **not** throw an exception for a missing ATM ID. It logs at level 3 and returns Flat. Every try/catch and every ...Safe() helper around these calls is therefore dead code for this particular failure — the only way to avoid the errors is not to make the call.

This is not a "zombie ID left over from a closed trade": that case is already handled correctly by ProcessNormalAtmTradeClose() and by Defense #3. It is purely a registration-window race on brand-new IDs.

### Fix applied

A new IsAtmIdQueryable() gate. An ID is only queried once the AtmStrategyCreate callback has confirmed registration, with a 2-second time-based fallback so Playback stays safe if the callback never fires. Inside the window the helper returns Flat — which is the same value NT8 was returning anyway, minus the log flood. Defense #3 still evicts genuinely dead IDs at 10 s.

```

private bool IsAtmIdQueryable (string strategyId)
{
    if (string.IsNullOrEmpty (strategyId)) return false;
    if (strategyId == atmStrategyId)
        return isAtmStrategyCreated
            || _atmIdsSetUtc == DateTime.MinValue
            || (DateTime.UtcNow - _atmIdsSetUtc).TotalSeconds >= AtmRegistrationQuietSeconds;
    if (strategyId == martingaleAtmStrategyId)
        return martingaleAtmStrategyCreated
            || _martingaleIdsSetUtc == DateTime.MinValue
            || (DateTime.UtcNow - _martingaleIdsSetUtc).TotalSeconds >= AtmRegistrationQuietSeconds;
    return true; // foreign/confirmed id
}

```

## Verification

Recompiled 29 July at 14:35:10 (NT8 only replaces NinjaTrader .Custom.dll on a successful build; zero compile errors logged). Both instances re-enabled at 14:36:44 and 14:37:35. The figures below cover the whole session that followed: **14:35 to 19:43, 6,041 log lines, 130 filled entries** (115 on Sim101, 15 on Sim102).

| Check                                    | Before            | After (130 entries)        |
|------------------------------------------|-------------------|----------------------------|
| ATM "does not exist" errors              | 6 to 12 per entry | 0 (approx. 900 expected)   |
| NT8 errors / warnings (level 2-3)        | present           | 0                          |
| Unwanted flatten / disable events        | 8 in 5 days       | 0                          |
| Bug #1 trigger: OCO stop quantity change | flatten + disable | 111 occurrences, no action |
| Bug #1 trigger: target partial fill      | flatten + disable | 125 occurrences, no action |

The last two rows are the strongest evidence. The exact state that killed Sim102 at 06:50:54 — a target partially filling and pushing its OCO sibling stop into Change submitted — recurred **111 times** during the session, with 125 target partial fills. Every single one passed without incident. This is not a theoretical verification: the defect was exercised repeatedly by live market conditions.

## Summary of code changes

The patch is deliberately additive. Across roughly 11,500 lines, **only 2 lines were removed**: the old NAKED POSITION DETECTED print (replaced by the SUSPECTED/CONFIRMED pair) and the OrderState.Submitted line of the condition rewritten in Bug #1. Everything else is insertion.

| Method                                   | Change                                             |
|------------------------------------------|----------------------------------------------------|
| field declarations                       | 4 new private fields + 2 constants                 |
| OnStateChange()                          | reset the 2 new flags on disable                   |
| OnExecutionUpdate()                      | 1 line: stamp _lastFillActivityUtc                 |
| OnMarkerAccountExecutionUpdate()         | 1 line: stamp _lastFillActivityUtc (ATM fills)     |
| TryGetAtmMarketPositionSafe()            | registration-window guard                          |
| GetAtmStrategyMarketPositionTickCached() | registration-window guard + new IsAtmIdQueryable() |
| CheckForNakedPositions()                 | grace period + double confirmation                 |
| HasWorkingProtectiveOrders()             | 4 additional order states accepted                 |

## No signal or trading logic was touched

To make this verifiable rather than assertable, every method in the file was extracted and hashed against the pre-patch backup. Result: **221 methods analysed, 8 differ** — exactly the eight listed above. All 28 methods on the signal and trading path are byte-identical:

OnBarUpdate · ComputeBarSignalSnapshot · UpdateNormalizedSignalSeriesAndDrawSignalArrows · BuildSignalTriggerName · CheckTradingTimeframes · CheckFlattenTimeframes · IsTimeInWindow · GetCurrentTradePosition · FlattenEverything · FlattenEverythingInternal · FlattenNakedAccountPosition · SubmitMartingaleRecoveryEntry · ShouldStartMartingaleRecovery · ResetMartingaleRecovery · ProcessNormalAtmTradeClose · ProcessMartingaleAtmTradeClose · ClearNormalAtmState · HandleAtmExecution · OnOrderUpdate · ManageNewsFilter · IsNewsTradingBlocked · ManageFixedBreakeven · ClearPendingSignalEntry · ClearPendingReverse · EvictStaleAtmIdsIfTimedOut · IsAtmMidTradeStale · ValidateAtmTemplate · ComputeAtmTradePnl

Note in particular that `FlattenNakedAccountPosition()` itself is unchanged — the patch changes *when* the watchdog calls it, never *what it does*. Likewise `AtmStrategyCreate` and its callback are untouched.

Semantically, both changed behaviours are strictly more permissive than before: the watchdog now flattens less often (never differently when the position is genuinely unprotected — only 5 s later), and the ATM guard returns `Flat`, which is the value NT8 was returning anyway inside that window, minus the log flood.

---

## Suggestions for the author (not implemented)

- **Reconsider `Account.Flatten()` in the emergency path.** Even with the false positives fixed, any future watchdog trip still silently disables the strategy. Closing with an explicit market order or `AtmStrategyClose()` would keep the strategy alive after an emergency flatten. Worth weighing against the ghost-reversal issues that presumably motivated the current choice.
- **Expose the new constants as user properties.** `NakedGraceSeconds (5)`, `NakedReCheckSeconds (5)` and `AtmRegistrationQuietSeconds (2.0)` are currently hard-coded. All measurements in this report come from the NinjaTrader internal simulator; a live broker connection will have a different and probably longer bracket latency tail, so these values should be re-measured before running the strategy on a funded account.
- **Consider logging the watchdog decision even when it does not fire.** The `SUSPECTED` line now gives an early warning of a widening race window before it becomes destructive.

Report generated 29 July 2026 from NinjaTrader 8 log and trace analysis. All timestamps are local platform time as recorded in `log.20260729.000000.en.txt` and `trace.20260729.000000.txt`. A pre-patch backup of the source file is preserved as `GodZillaKilla.cs.bak_20260729_nakedwatchdog`.